

REFERENCE EXAMPLE

Weekly Competition #7: The Operator

Don't build this operator. Use this brief as a template for how to scope your own.

The point of the comp is to pick an operational workflow YOU would actually use, then build a folder that turns Claude into the operator who handles that workflow end-to-end. This example shows what good scope looks like, what real decision logic sounds like, and how to translate it into a folder-based specialist.

Read it. Steal the structure. Build your own.

THE EXAMPLE CLIENT

My name's Devon. I run a small technical recruiting firm out of Denver. Four recruiters including me. We specialize in placing engineers at early-stage startups, mostly Series A and B, mostly remote-friendly roles.

Our reputation is built on speed and signal. Founders come to us because we don't waste their time. They get 3 to 5 candidates per role, every one of them already screened, every one of them a real maybe.

THE PROBLEM

Resumes come in all day. From our website, from LinkedIn, from referrals. Most are wrong fit. Some are right on. The problem is figuring out which is which fast enough that I'm not spending half my day on triage instead of actual recruiting.

Right now I read every inbound. That's 40 to 60 resumes a day. I make a call in about 90 seconds: book a call, send a polite decline, or save for a future role. The call itself isn't hard. The volume is what kills me.

I don't need an applicant tracking system. I have one. I need a first-pass screener that reads the resume, makes the same decision I'd make, and either drafts a "let's talk" email or a gracious decline. The screener has to handle the routing. I just want the output.

WHAT DEVON NEEDS

An AI operator I can paste a resume into. It reads the resume against my current open roles. It outputs one of three things: a drafted outreach email if the candidate is a yes, a drafted decline if the candidate is a no, or a flag for my review if the candidate is a maybe.

The decision logic has to match how I actually think. Specific tech stack matters less than years of experience at a similar stage company. Job hopping is a yellow flag, not a red one. A senior

engineer applying for a mid-level role is usually trying to downshift on responsibility, which can be a fit for some of my clients. Edge cases like that are where most tools fail.

WHAT GOOD SCOPE LOOKS LIKE

Notice what Devon is specific about:

- A specific workflow (resume → decision → drafted response)
- A specific output (one of three things, every time)
- Specific decision criteria (stage experience > tech stack match)
- Specific edge cases (the senior applying mid-level)
- Specific escalation rule (when in doubt, flag for human review)

Your workflow should be at least this specific. "Customer support" is too broad. "Refund request triage for an ecommerce store doing under 200 orders/month" is the right level.

THE ANGLE FOR THIS WEEK

An operator is not a chatbot.

A chatbot responds. An operator makes a decision, then routes the work. Your folder has to encode that decision logic explicitly. What are the criteria? What are the edge cases? What gets escalated? What gets handled automatically?

When the operator gets an input, it shouldn't ask the user what to do. It should decide and act. That's the whole point. The user comes back to find the work already done, or correctly flagged for their input.

THE METHODOLOGY

This is interpretable context methodology. Folders as architecture. Each file does one job well. The structure tells you what's where.

Your operator folder must contain these five things:

```
operator-name/  
  ■■■ identity.md  
  ■■■ rules.md  
  ■■■ examples.md  
  ■■■ reference/  
  ■ ■■■ (criteria checklists, templates, rubrics)  
  ■■■ README.md
```

WHAT GOES IN EACH FILE

identity.md — Who is this operator and what workflow do they own? Be explicit about what falls inside their job and what falls outside it.

rules.md — The decision logic. Criteria. Edge cases. Escalation rules. This is the heart of the operator. Be specific. "Use good judgment" is not a rule. "If years of experience is less than 3, decline unless the candidate has shipped a notable open source project" is.

examples.md — Two or three example decisions showing the logic in action. Include at least one edge case. Show the operator handling something the obvious rules don't cover.

reference/ — Sub-folder for any criteria checklists, templates, scoring rubrics, or response templates the operator needs to do its job.

README.md — How someone uses this folder. One paragraph someone could follow with no other context.

THE BAR

Someone with no context should use your folder, feed it three real examples, and trust the output enough to act on it.

If the operator kicks the decision back to the user every time, you've missed the assignment.

YOUR DELIVERABLE

A public GitHub repo containing your operator folder. Drop the link in the comments along with two or three sentences on what workflow your operator handles and what kind of decisions it makes.

No GitHub? Use a Google Drive folder for now.

WHAT WE'RE LOOKING FOR

Four things, in this order:

1. Does the operator actually make decisions? Or does it kick the question back?
2. Are the edge cases handled or hand-waved?
3. Would the output be trustworthy enough to act on?
4. README quality. Can a stranger figure this out?